SEMANA 4 · PARTE 2

Ivett Anahi Zaragocin Mora

Interrupciones externas en ESP32

ISR · IRAM_ATTR · VOLATILE

Bloque 5 · La ISR

¿Qué es una ISR? ISR = **Interrupt Service Routine** : la función que **atiende el aviso** (el *callback* de la semana 3).

FUNCIÓN NORMAL

La llamas tú, **donde decides**.

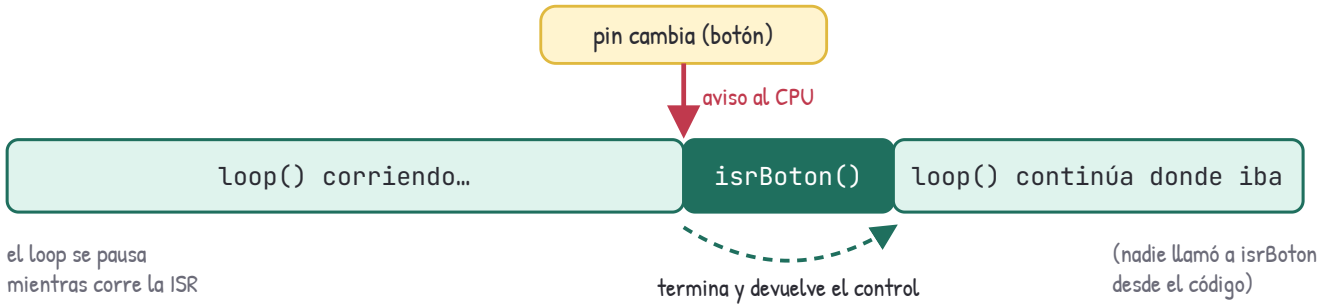
```
saludar();   // la llamo yo
```

ISR

La llama el **hardware**, **en cualquier momento**.

```
void isrBoton() { /* la llama el HW */ }
```

Flujo de ejecución con una interrupción



Regla 1: la ISR debe ser corta

✔ Sí va en la ISR

- Subir una **bandera**
- Guardar el tiempo: `millis()`
- Sumar 1 a un contador
- `digitalWrite` (es rápido)

✘ No va en la ISR

- `Serial.print` / `println`
- `delay()`
- Cálculos largos
- Esperar a que algo pase

Si la ISR tarda: el watchdog

ISR larga
Serial, delay...

→

Watchdog
detecta que algo está trabado

→

Reinicio
el ESP32 arranca de nuevo

→

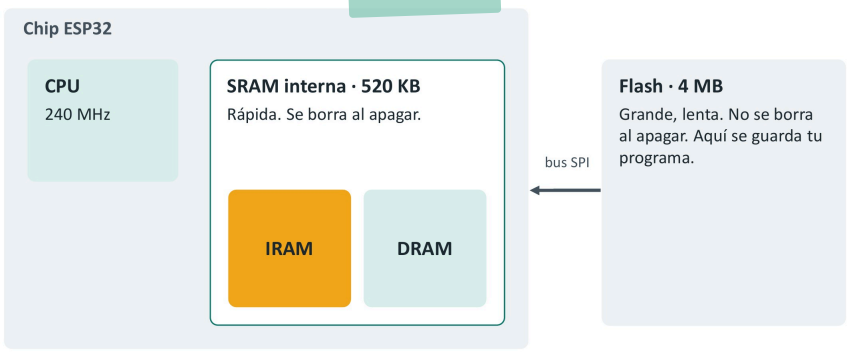
Monitor Serie
"Guru Meditation Error"

Cadena de fallo de una ISR larga

Síntoma: **"el ESP32 se reinicia de vez en cuando"**. Revisa si hay `Serial` o `delay` en una ISR.

Bloque 6 · IRAM

CPU 240 MHZ **SRAM 520 KB** **FLASH 4 MB**



Dentro del chip ESP32 y la flash externa (bus SPI)

Memoria	Rasgos
SRAM interna 520 KB	Rápida. Se borra al apagar.
Flash 4 MB	Grande y lenta. No se borra. Aquí se guarda el programa. Se accede por el bus SPI.

✦ IRAM y DRAM: dos zonas de la misma SRAM

IRAM · Instruction RAM

Guarda **instrucciones**: código que la CPU ejecuta.
Aquí pondremos la ISR.

DRAM · Data RAM

Guarda **datos**: por ejemplo, las variables globales.
Aquí vive la variable `alerta`.

✦ La caché: leer rápido de una flash lenta

CPU
pide la siguiente instrucción

→

Caché
copia de lo leído hace poco

→

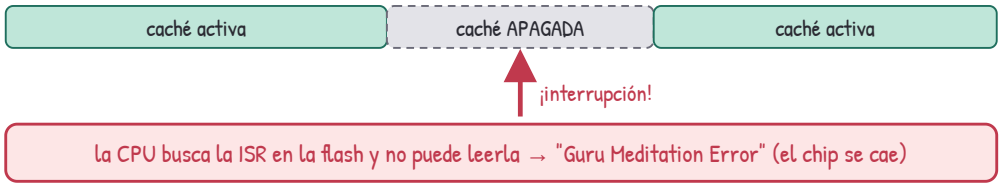
Flash
el programa completo, lento

✦ El problema: a veces la caché se apaga

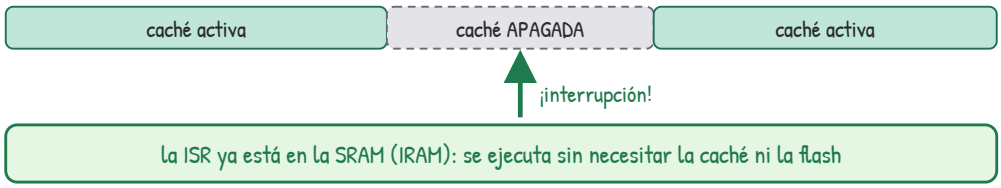
1. **El ESP32 escribe en la flash**
   por ejemplo, al guardar datos.
2. **La caché se apaga**
   un momento.
3. **Llega una interrupción**
   y la ISR está en la flash.
4. **La CPU no puede leerla**
   el chip se cae.

Misma interrupción, dos sitios para la ISR

✘ ISR en flash (sin IRAM_ATTR)



✔ ISR en IRAM (con IRAM_ATTR)



La ISR es código de emergencia: debe estar siempre en IRAM.

✦ IRAM_ATTR: "guarda esta función en la IRAM"

```
void IRAM_ATTR isrBoton() {
  // código corto de emergencia
}
```

↑ VA ENTRE EL TIPO Y EL NOMBRE

Regla Toda ISR lleva `IRAM_ATTR`. Sin él, falla de vez en cuando.

```
// ✘ ISR en flash: falla de vez en cuando
void isrBoton() {
  alerta = true;
}
```

```
// ✔ ISR en IRAM: siempre disponible
void IRAM_ATTR isrBoton() {
  alerta = true;
}
```

Bloque 7 · volatile

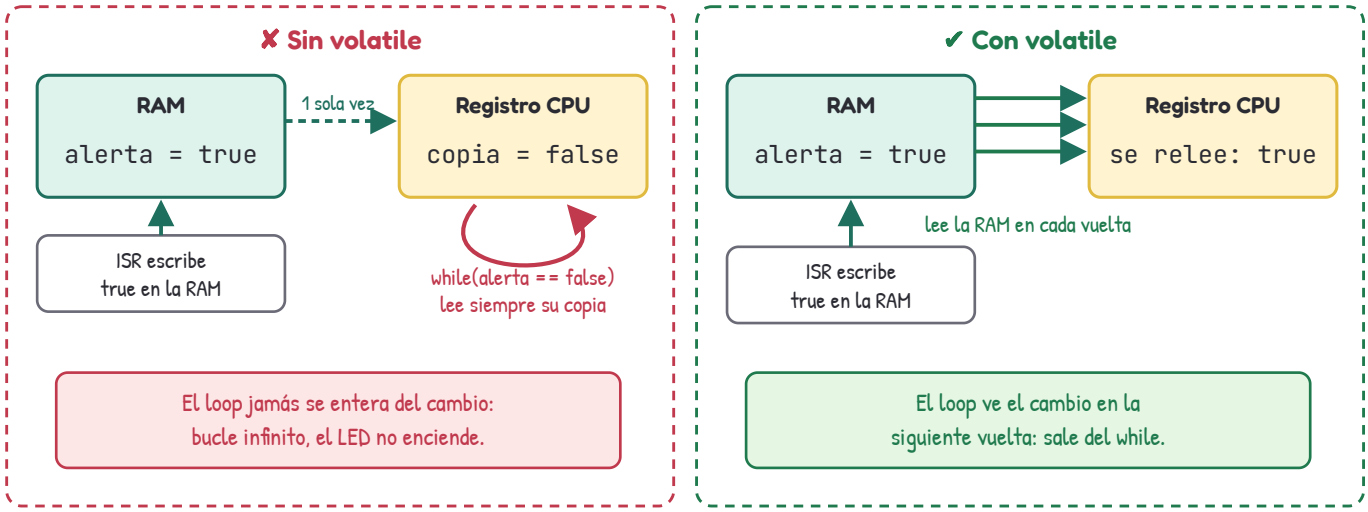
RAM	Registro de la CPU
Donde vive la variable `alerta`. La ISR y el `loop()` la pueden ver.	Casillas dentro de la CPU: para usar un dato, primero **se copia aquí**.

✦ El compilador optimiza

```
while (alerta == false) {
  // nada aquí adentro
  // cambia alerta
}
```

El compilador piensa:
"Nadie cambia `alerta` aquí: la copio una vez al registro."

El problema: el compilador **no sabe que existe la ISR** (la dispara el hardware).



Qué pasa con la variable compartida con y sin volatile

✦ Lo que hace volatile

```
volatile bool alerta = false;
```

Significa: la variable puede cambiar sin aviso; **léela de la RAM cada vez**. Solo eso: no la hace más rápida ni más segura.

Actividad: ¿va volatile?

La prueba: ¿la usan la ISR **Y** el loop?

Variable	¿La toca la ISR?	¿La toca el loop?	¿volatile?
`alerta`	escribe	lee	Sí
`pulsos`	escribe	lee	Sí
`tUltimo`	lee y escribe	—	No
`total`	—	lee y escribe	No

No importa dónde se escribe, sino cuántas partes del programa la tocan.

⚠ Con un `println` en el loop a veces parece funcionar sin `volatile`; al quitarlo, falla. Si cumple la regla, **va volatile**.

Resumen rápido

- **ISR**: la llama el hardware, nunca tú. Debe ser **corta** (bandera, `millis()`, contador); sin `Serial` ni `delay`, o el **watchdog** reinicia el ESP32.
- **IRAM_ATTR**: guarda la ISR en la IRAM para que funcione aunque la **caché** esté apagada. Toda ISR lo lleva.
- **volatile**: obliga a leer la variable de la RAM cada vez (sin copia vieja en el registro).
- Regla: si la **ISR y el loop** tocan la variable → `volatile`.